World Electric Vehicle Journal

MDPI

Article

An Efficient Drowsiness Detection Framework for Improving Driver Safety Through Supervised Learning Models

Hassan Harb

College of Engineering and Technology, American University of the Middle East, Egaila 54200, Kuwait; hassan.harb@aum.edu.kw

Abstract

Nowadays, we live in the smart mobility era in which vehicles are equipped with small sensing devices to collect various road information. With such sensors, we are able to provide an overview of what is happening on the road and offer an efficient solution for transport problems such as congestion, accidents, avoiding traffic lights, fuel consumption, etc. Particularly, driver drowsiness is one of the most important problems that transportation systems face and mostly leads to severe accidents, injuries, and deaths. In order to overcome such a problem, a set of sensor devices has been integrated into vehicles to monitor driver and driving behaviors, and then to evaluate the driver's situation, e.g., drowsy or awake. Unfortunately, most of the proposed drowsiness detection techniques are dedicated to analyzing one behavior type, but not both, which may affect the accuracy rate of the detection. In this paper, we propose an efficient drowsiness detection framework (RDDF) that may analyze one behavior or be adapted to both of them in order to increase the accuracy of drowsiness detection. Mainly, RDDF periodically monitors the driver and driving behaviors, extracts important patterns, and then uses and compares a set of supervised learning models to detect drowsy drivers. After that, RDDF proposes a modified version of the K-nearest neighbors (KNN) model called Jaccard-KNN (JKNN) that increases drowsiness detection accuracy and overcomes several challenges imposed by traditional models. The proposed framework has been preliminarily validated through real sensor data, and we show the effectiveness of our framework in detecting real-time drowsy drivers with an accuracy rate of up to 99%.

Keywords: intelligent transportation systems; road safety; drowsiness detection; driving driver behaviors; supervised learning models

check for updates

Academic Editors: Xiang Dong, Jiajia Chen and Biao Yu

Received: 31 August 2025
Revised: 18 October 2025
Accepted: 27 October 2025
Published: 13 November 2025

Citation: Harb, H. An Efficient Drowsiness Detection Framework for Improving Driver Safety Through Supervised Learning Models. *World Electr. Veh. J.* **2025**, *16*, 620. https://doi.org/10.3390/wevj16110620

Copyright: © 2025 by the author. Published by MDPI on behalf of the World Electric Vehicle Association. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Mobility is a critical element of our society. Thus, most governments and societies give significant attention to building an efficient transportation system that ensures faster and safer mobility of their citizens. Recently, the emergence of smart sensing devices is set to push the transportation systems to the next level, mostly known as smart mobility (SM) or intelligent transportation system (ITS), and it has been a driving force behind the increase in the traditional system performance and safety at a lower cost [1,2]. With the help of sensors, the sensing technology allows government and authorities to obtain live data about traffic roads, and then make accurate and quick decisions about any event that has occured or an abnormal situation. Subsequently, traffic data collection benefits include the reduction in traffic congestion and saving traveler time, identification of vehicles disobeying a speed limit or red light, improvement in traffic safety, rapid detection of

accidents, reduction in infrastructure damage, parking management, and vehicle violation detection [3,4]. Therefore, it becomes essential to deploy and enable new services and applications that take advantage of the collected traffic data in order to optimize and increase the efficiency of the transportation systems.

In this work, we are interested in driver drowsiness detection as one of the most important safety issues studied in ITS. Unfortunately, the number of accidents, along with the resulting injuries and deaths because of the drowsiness problem, has increased over the last years with more than 1.3 million accidents per year [5]. This is due to many factors that contribute to such a problem, such as depression, sleep disorders, shift workers, long driving time, road monotony, medication, etc. [6,7]. Indeed, sensing-based technology offers two categories of sensors to detect the drowsiness problem: driving-based and driver-based. The first category monitors driving behavior such as vehicle lane deviation, steering wheel movement, and vehicle speed, whilst the second category measures the driver's behavior such as eye and head movements, facial expression, and organ activities through several physiological signals such as Electroencephalography (EEG), Electrooculography (EOG), and Electrocardiography (ECG). Thus, after collecting driver behavioral measures, researchers mainly use machine or deep learning models to determine the driver's situation as awake or drowsy. However, most of the proposed techniques suffer from three main drawbacks [8,9]: First, they are dedicated to analyzing one behavioral category or, sometimes, limited to one categorical measure but they cannot be generalized to all of them. Second, the accuracy of the drowsiness detection is highly related to the tested dataset; thus, the overfitting problem is very likely to happen. Third, they mostly give a lower performance in terms of precision and F-measure when dealing with an imbalanced data problem.

In this paper, we propose an efficient design that analyzes one or several behavioral categories in order to detect drowsy drivers while ensuring a high accuracy level. The proposed design is called RDDF, which periodically monitors various measures for driving and driver behaviors for a later analysis and decision-making. Indeed, RDDF is mainly two-fold: First, it tests and compares the performance of various supervised learning models in terms of detecting the drowsiness problem, such as logistic regression (LR), random forest (RF), decision tree (DT), naïve Bayes (NB), K-nearest neighbors (KNN), and multi-layer perceptron (MLP). Second, it proposes JKNN that increases the accuracy of drowsiness detection as well as overcoming several challenges imposed by traditional models.

The remainder of this paper is organized as follows: Section 2 presents an overview of various driving and driver behavioral analysis techniques related to drowsiness detection existing in the literature. Section 3 describes the design of RDDF while detailing each of its phases. Section 4 describes the implementation of RDDF and explains the obtained results. Finally, Section 5 concludes the paper and gives directions for future work.

2. Related Work

In the literature, one can find a wide range of techniques dedicated to addressing the driver drowsiness problem [10,11]. Mainly, researchers have focused either on driving-related or driver-related measures to analyze the current behavior and determine the driver's situation. The authors of [12] provide an overview of various measures and products used to detect drowsy drivers, while those of [13,14] present a comprehensive survey of different learning models used for driver behavior analysis.

On the one hand, researchers of [15–18] proposed drowsiness detection techniques based on driving behavior analysis. The authors of [15] introduced a novel method based on vehicle speed measurements to recognize risky driving behavior. First, the proposed method searches for the key distribution pattern of each driver that corresponds to the

positive speed-change ratio within various speed ranges. Then, it uses the support vector machine (SVM) model to determine risky drivers based on the overall patterns of all drivers. In [16], the authors proposed a feature- and prediction-based approach to detect drowsy drivers in three main steps: The first step periodically monitors driving behavior based on acceleration, braking, and steering wheel metrics. The second step extracts four features (i.e., mean, variance, entropy, and power) from each collected data set, and then assigns a weight to each measure based on the Pearson correlation coefficient. The last step applies two predictive models, e.g., classification and regression, where a decision tree and an ensemble classifier are used to evaluate the performance. The authors of [17] introduced a feature-based selection method based on steering wheel movement for driver drowsiness detection. First, the technique applies four filter indexes (i.e., Fisher, correlation, t-test, and mutual information) to extract time- and frequency-domain features from steering wheel records. Then, the most important features are selected according to a fuzzy inference system before applying the SVM model to classify the driver's situation as awake or drowsy. In [18], the authors proposed a convolutional neural network (CNN)-based model to detect driver behavior using driving signals such as gravity, speed, throttle, acceleration, and revolutions per minute (RPM). The proposed model first converts the temporal dependencies and overlapping time windows of signals into images through the recurrence plot (RP) technique. Then, the images are passed to the CNN model to classify the driver's state into five types: normal, aggressive, distracted, drowsy, and drunk.

On the other hand, some works such as [19–22] are dedicated to driver behavior-based analysis for drowsiness detection. In [19], the authors proposed an efficient technique for driver sleepiness detection that combines wavelet transform (WT) and long short-term memory (LSTM) models. The intuition of the technique is based on the fact that EEG and EOG signals vary correspondingly when alpha phenomenon occurs. Hence, the authors used the wavelet transform to calculate the variation in EEG and EOG signals in the time and frequency domains, followed by the LSTM model to classify various waveforms into sleep or alert states. The authors of [20] proposed PCANet, which integrates principal component analysis (PCA) and a deep learning framework for efficient driver fatigue detection. First, PCA is used to preprocess multi-channel EEG signals and reduce the data dimensionality in order to overcome the dimensionality explosion problem. Then, the SVM and KNN models are employed to classify the EEG signals and determine the fatigue level. In [21], the authors proposed an adaptive multi-scale entropy technique based on the EEG data for fatigue-related driving detection. The proposed technique combines an adaptive scaling factor (ASF) and an entropy-based feature extraction method; ASF is used to scale down the length of the signal, while the entropy method is used to extract signal features for classification.

Finally, a few studies such as [23,24] proposed hybrid solutions that analyze both driving and driver behaviors to detect fatigue states. In [23], the authors developed two CNN models to detect the degree of drowsiness every minute and to predict moderate drowsiness, respectively. Both models rely on the analysis of physiological and behavioral measures such as heart rate, head and eye movements, lane deviation, speed, steering wheel angle, etc. The authors of [24] introduced a mathematical model based on the cumulative time effect to monitor the development of the drowsiness state. The proposed model defines a mixed-effect ordered logit function and an increasing temporal function to evaluate the variation in lane deviation and driver eye closure as a function of time. Consequently, it detects drowsy drivers when an abnormal variation is observed.

3. RDDF: Design Description

The flowchart shown in Figure 1 presents the general process of RDDF, starting from the driver data acquisition to the drowsiness detection phase. Mainly, the proposed framework periodically monitors driving and driver measures, and then generates a dataset composed of shapelets that represent the driver's state. After that, the dataset is split into training and testing sets before applying a set of classification models, including the newly proposed model, to determine the driver's state, e.g., alert or drowsy. Finally, the driver's database is updated according to the driver's behavior. In the following sections, we describe each step of the proposed framework in detail.

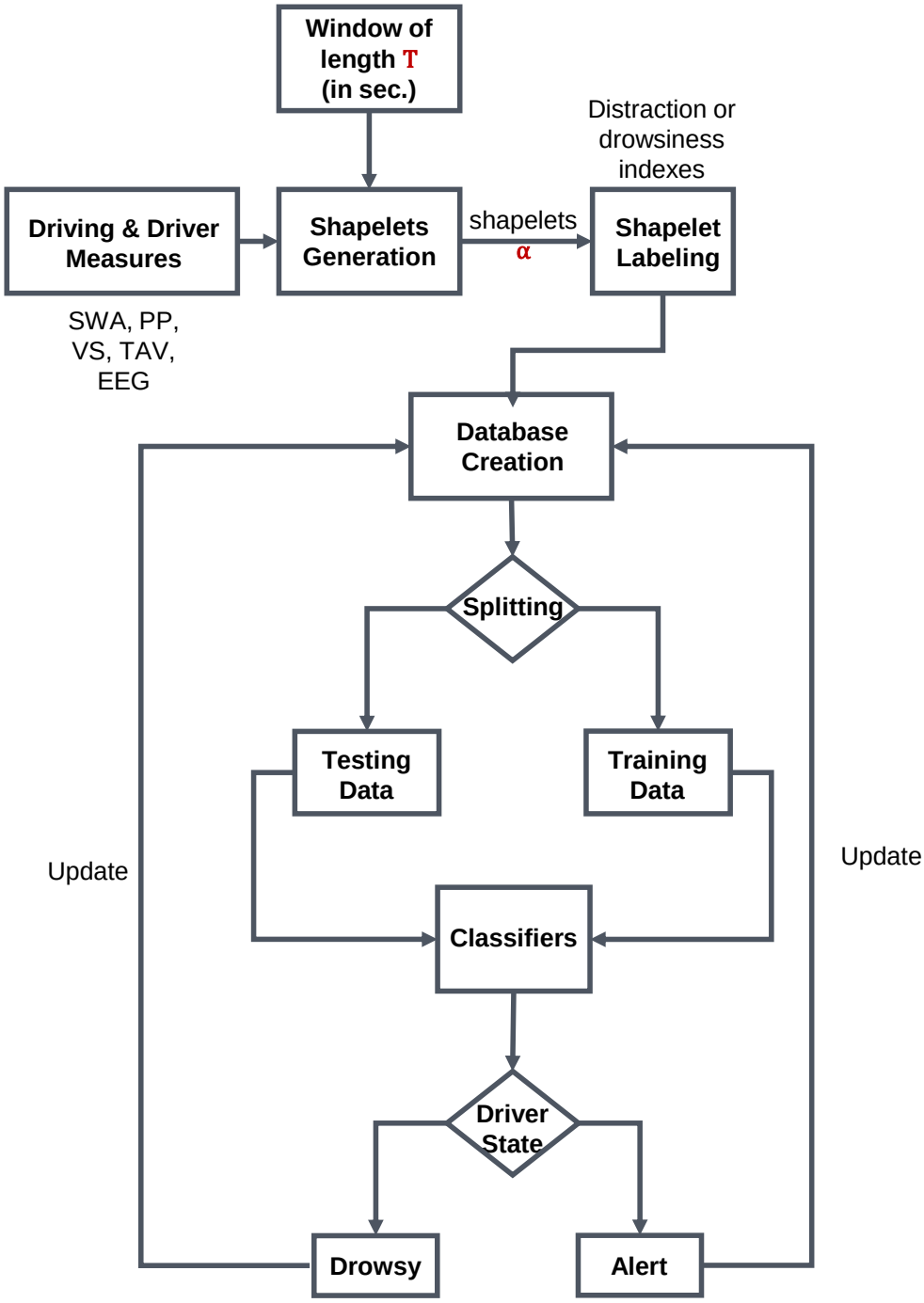

Figure 1. Flowchart of RDDF design.

3.1. Measured Data Acquisition

The first step in our framework is to collect data on various driver and driving measures. Indeed, the data acquisition phase consists of three processes: experiment and simulation setup, shapelet generation, and shapelet labeling.

3.1.1. Experiment and Simulation Dataset Description

On the one hand, we used experimental data from 40 subjects collected in real driving and simulator environments within the VTD project [25] (Figure 2). The VTD driving dataset includes moment-by-moment binary annotations (0 = alert; 1 = drowsy) produced by the VTD data collection team; these annotations are supplied with the dataset and were used as the ground truth for all driving-based measures in our experiments. The VTD annotations were obtained by the dataset authors through synchronized behavioral/video review and signal monitoring during the driving and simulation sessions; we used these author-provided labels directly, with no further re-labeling. For each subject, we focused, in this study, on four attributes that capture driving behavior: steering wheel angle (SWA), pedal pressure (PP), vehicle speed (VS), and transverse angular velocity (TAV). Each subject's recordings are segmented into fixed-length windows, and the corresponding features are calculated in both mean and standard deviation forms, resulting in synchronized time series suitable for drowsiness analysis. More formally, we considered that the raw dataset of VTD experiments, D_E, is defined as $D_E = \{F_1^E, F_2^E, \ldots, F_{40}^E\}$, where each file F_i^E corresponds to one subject and contains a multivariate time-series representation of the four attributes, e.g., $F_i^E = \{A_1, A_2, \ldots, A_4\}$. On the other hand, we used a publicly available EEG driver drowsiness dataset described in [26]. This EEG dataset includes per-sample binary labels (0 = awake; 1 = drowsy), which we also used directly as ground truth for the EEG measure. The data was collected using an Emotiv EPOC+ headset from a single EEG channel, and then preprocessed to filter and remove noise before storage. The dataset consists of 17 CSV files, where each file contains around 115,200 readings; 52.9% of those readings are labeled as drowsy, while the remainder represent the awake state. Similarly, we considered that the raw dataset of simulations, D_S, is represented as follows: $D_S = \{F_1^S, F_2^S, \ldots, F_{17}^S\}$, where each file F_i^S contains only one attribute, e.g., the EEG signal.

3.1.2. Shapelet Generation

As already mentioned, our proposed framework is flexible and can detect driver's drowsiness based on driving or driver behavior separately or simultaneously. In this paper, we defined the following set of measures: $\mathcal{M} = \{SWA, PP, VS, TAV, EEG\}$. Subsequently, we adopted a periodic model to analyze the data of each measure and detected the driver's drowsiness state. In this model, each measure is monitored for a fixed window of length T seconds, where the collected data are represented by a vector of readings called a shapelet, defined as follows: $H_i^j = \{m_1, m_2, \ldots, m_T\}$, where T is the total number of readings collected during the window and j corresponds to the measure $M_j \in \mathcal{M}$. Thus, the entire data collected for each measure M_j in an experiment or a simulation file (e.g., F_i^E or F_i^S) are considered as a set of shapelets $\mathcal{H}^j = \{H_1^j, H_2^j, \ldots, H_\alpha^j\}$, where α is the total number of shapelets generated for the measure M_j.

3.1.3. Independence and Coupling Analysis

Since the indicators originate from distinct sensing modalities (vehicle dynamics and EEG), they are expected to capture complementary aspects of the driver's state rather than redundant information. To verify this assumption quantitatively, we computed the pairwise Pearson correlation matrix among the four driving indicators (*SWA*, *PP*, *VS*, and *TAV*) and the EEG-derived drowsiness index (Table 1). All the absolute correlation coefficients were

below 0.45, indicating only weak to moderate correlations between variables. To further assess potential multicollinearity, we calculated the Variance Inflation Factor (VIF) for each indicator, and all the values were less than 2, confirming the absence of significant collinearity. Given this low interdependence, no explicit de-correlation or dimensionality reduction (e.g., PCA) was required prior to the shapelet generation.

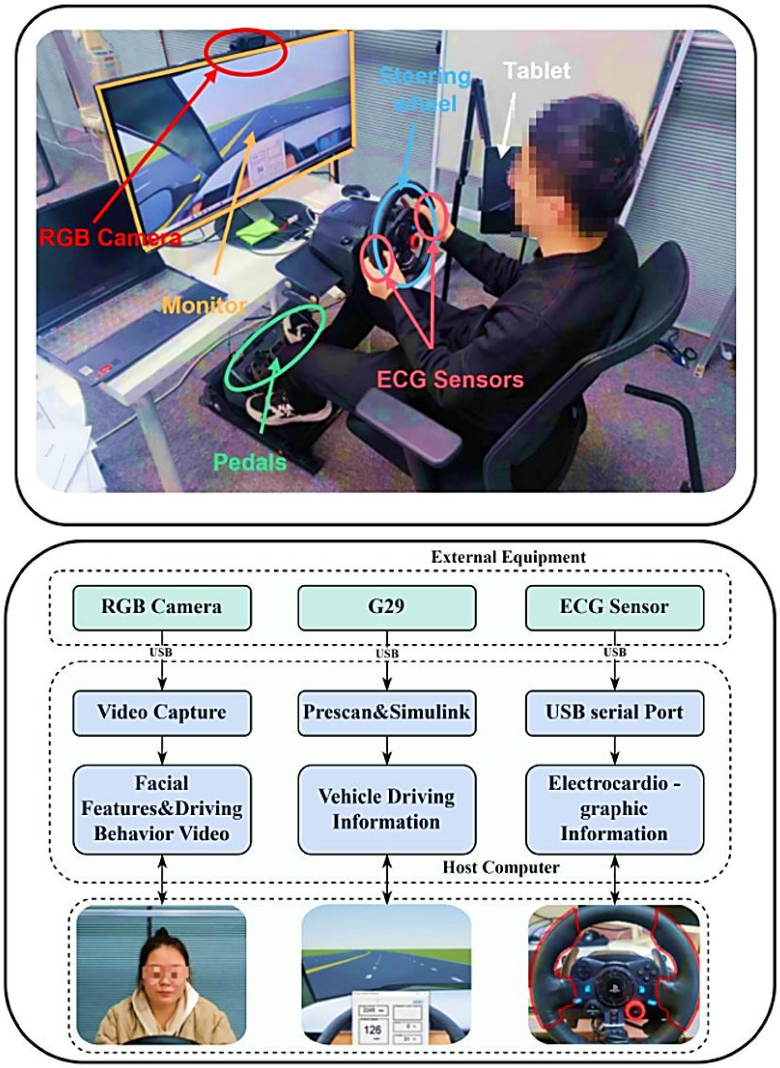

Figure 2. VTD simulator [25].

Table 1. Pairwise Pearson correlation coefficients among sensor indicators.

	SWA	PP	VS	TAV	EEG
SWA	1.00	0.32	0.27	0.18	0.21
PP	0.32	1.00	0.35	0.42	0.29
VS	0.27	0.35	1.00	0.44	0.26
TAV	0.18	0.42	0.44	1.00	0.19
EEG	0.21	0.29	0.26	0.19	1.00

3.1.4. Shapelet Labeling

After generating shapelets for each measure, the last step of this phase aims to assign a label to each shapelet indicating the driver's situation. The idea behind shapelet labeling is to allow supervised learning models to recognize or classify the driver's state, either drowsy or awake. In our framework, the shapelet labeling process is defined such that a

shapelet H_i^j is labeled as 1 (i.e., drowsy state) if all its readings indicate a drowsy state, and 0 otherwise. In the VTD dataset, the driver's state is explicitly annotated using a binary label. A value of 0 indicates an alert (normal) state, whereas 1 corresponds to a drowsy state at each recorded moment. Similarly, in the EEG dataset, every signal sample is already labeled as 0 (awake) or 1 (drowsy) by the dataset provider. Therefore, we assumed that each shapelet H_i^j has its corresponding label $L_i^j \in \{0, 1\}$. Formally, if L_t denotes the provided label (0/1) at timestamp t inside the window, then

$$L_i^j = \begin{cases} 1, & \text{if } \forall t \in \text{window}_i,\ L_t = 1, \\ 0, & \text{otherwise.} \end{cases}$$

We chose this "all-samples" rule because it reduces the occurrence of spurious (single-sample) positive labels and helps prevent excessive false alarms when using short windows (T = 2–4 s). To assess the sensitivity of our results to this choice, we additionally tested two alternative aggregation rules: a majority vote rule and a $\geq$50% threshold. However, the conclusions about relative classifier performance and the superiority of the JKNN model remain robust across these aggregation choices. At the end of this phase, a database $\mathcal{D}$ is created, containing the shapelet sets for all measures in $\mathcal{M}$: $\mathcal{D} = \{\mathcal{H}^1, \mathcal{H}^2, \ldots, \mathcal{H}^5\}$, where $\mathcal{H}^j$ corresponds to the measure $M_j \in \mathcal{M}$.

3.2. Data Classification and Drowsiness Detection

The second phase of our framework aims to classify and detect the driver's state through a set of supervised learning models. Subsequently, this phase consists of three main processes that are described in the following sections.

3.2.1. Database Division

For performance evaluation purposes, we divided the database $\mathcal{D}$ into two partitions: training data, which consist of β% of the overall data, and testing data, which consist of the remaining portion. The division process adopted in our framework is mainly based on the subjects instead of the shapelets. For instance, when $\beta = 50$, half of the files in D_E and D_S corresponding to the subjects are considered as training data, and the other half are used for the testing process. Consequently, we consider that the shapelet sets for measures corresponding to the training data are represented by $\mathcal{D}^{tr} = \{\mathcal{H}_{tr}^1, \mathcal{H}_{tr}^2, \ldots, \mathcal{H}_{tr}^5\}$, and those corresponding to the testing data are represented by $\mathcal{D}^{te} = \{\mathcal{H}_{te}^1, \mathcal{H}_{te}^2, \ldots, \mathcal{H}_{te}^5\}$.

3.2.2. Classifiers Description

Indeed, researchers have proposed many classification methods, each with its own process and performance, and none of them can be regarded as universally optimal for all applications. In this section, we applied a set of supervised methods to the testing data in order to identify the driver's state. Indeed, one can find a wide range of supervised models in the literature, each with its own process and performance. Thus, our objective is to apply several classifiers rather than focusing on a single one, and subsequently select the model that achieves the best performance in detecting driver drowsiness. In this work, we tested six standard classifiers, followed by the proposal of a new model, which are described as follows:

- Logistic Regression (LR) [27]: It is a statistical model that employs a logistic function combined with stochastic gradient descent to predict the class category of a given feature set. LR has been widely tested in numerous studies to detect driver drowsiness [28,29]. In this paper, we used the default parameters of LR provided by the scikit-learn library.

- Random Forest (RF) [30]: This is an ensemble learning method that can be efficiently applied for both classification and regression based on multiple hierarchical decision trees during the training phase. Subsequently, the classification output of the RF is the class selected by the majority of trees, while the regression output is the average value of all tree predictions. RF has also demonstrated its potential in outperforming the accuracy of the standard decision tree classifier for driver drowsiness detection [31,32]. In this work, the default values of the random forest classifier implemented in `scikit-learn` are considered, where the number of trees is fixed to 100.
- Decision Tree (DT) [33]: It is a decision support tool based on a tree structure and if–then–else decision rules. DT is commonly used in various operations research and decision analysis domains, particularly for driver drowsiness detection [34–36]. The decision tree algorithm is implemented in the `scikit-learn` library, where the default parameters, such as tree depth and feature index, are considered in this paper to automatically select the best decision based on the feature set.
- Naïve Bayes (NB) [37]: It is a probabilistic classifier based on Bayes' theorem with strong independence assumptions between features. NB has demonstrated its computational efficiency in various domains, including drowsiness detection [38,39]. The `scikit-learn` library provides several models for Naïve Bayes implementation, of which we used the Gaussian model in this work.
- K-Nearest Neighbors (KNN) [40]: It is one of the simplest learning classifiers that does not require creating a training model. The core of KNN mainly depends on measuring the distance between the training and testing data. Furthermore, KNN has shown strong performance in detecting drowsiness through a large number of studies [41,42]. By default, scikit-learn uses KNN with $K = 5$ and the Minkowski distance with uniform weights.
- Multi-Layer Perceptron (MLP) [43]: It is a supervised learning method based on an artificial neural network and the backpropagation algorithm for training. Mainly, MLP consists of three layers (input, hidden, and output) and uses an activation function to determine the predicted class. Furthermore, MLP is widely used in drowsiness detection [44,45], and we set the number of neurons in scikit-learn to 500, 250, and 40 for the three layers, respectively.
- Jaccard-based K-Nearest Neighbors (JKNN): In general, KNN uses a distance metric, such as Euclidean, to measure the difference between independent vectors of testing and training data. However, a distance metric is usually appropriate for data that do not exhibit direct correlation or temporal factors among their values. Hence, it is often preferable to use similarity metrics in applications where the information is temporally organized, thus considering correlation in the similarity measure proves to be more appropriate and can even yield more accurate results. In this work, we focused on the Jaccard coefficient to gauge the similarity and diversity between sample sets, as it has been used in a wide range of applications including community detection in social networks [46], document and web page plagiarism detection [47], attack detection [48], and market analysis [49]. Thus, the Jaccard similarity $Jac(H_i^j)$ between two shapelets $H_i^j \in \mathcal{D}^{tr}$ and $H_k^j \in \mathcal{D}^{te}$ that correspond to the same measure j is calculated as follows:

$$Jac(H_i^j, H_k^j) = \frac{|H_i^j \cap H_k^j|}{2 \times T - |H_i^j \cap H_k^j|} \tag{1}$$

Then, the same concept of KNN is applied, but in this case, the nearest shapelets are selected according to the Jaccard similarity measure, with the number of neighbors set to $K = 5$.

3.2.3. Drowsiness Detection

In order to detect driver drowsiness, we applied classifiers based on the generated shapelets instead of applying them directly to each record, as is commonly done in most existing works. In addition, RDDF is highly flexible, as it can detect driver drowsiness based on a single measure, either driving or driver behavior, or on multiple measures, such as both.

- Single-Measure Drowsiness Detection: In this scenario, we applied each classifier based on each measure $M_j \in \mathcal{M}$ to analyze the driver or driving behavior. Algorithm 1 shows the process of single-measure drowsiness detection, which takes as input the training shapelets of the monitored measure along with the desired classifier, and produces as output the driver's state according to the testing shapelets. The process starts by defining a set that stores all driver states of the testing shapelets, e.g., *states_set* (line 1). Then, for each shapelet in the testing set, its corresponding readings are considered as feature vectors before applying the selected classifier (lines 2–4). Once the predicted state is returned by the classifier, e.g., L_i^j, it is added to the list of driver states (lines 4–5). Finally, the performance metrics (accuracy, precision, recall, and F1) of the classifier are calculated (see Equations (2) to (5)), and the driver state is returned (lines 7–14).

Algorithm 1 Single-Measure Drowsiness Detection Algorithm

Require: A measure: $M_j \in \mathcal{M}$; Training data of a measure: $\mathcal{H}_{tr}^j \in \mathcal{D}^{tr}$; Testing data of a measure: $\mathcal{H}_{te}^j \in \mathcal{D}^{te}$; A classifier: C.
Ensure: Driver state: awake or drowsy.

1: $states_set \leftarrow \emptyset$
2: **for** each $H_i^j \in \mathcal{H}_{te}^j$ **do**
3: consider each $m_k \in H_i^j$ as a feature
4: $L_i^j =$ apply_classifier (C,H_i^j,$\mathcal{H}_{tr}^j$)
5: $states_set \leftarrow states_set \cup \{L_i^j\}$
6: **end for**
7: [accuracy, precision, recall, F1] = confusion_matrix ($states_set$, $\mathcal{H}_{te}^j$)
8: **for** each $L_i^j \in states_set$ **do**
9: **if** $L_i^j = 0$ **then**
10: **return** awake
11: **else**
12: **return** drowsy
13: **end if**
14: **end for**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{2}$$

$$Precision = \frac{TP}{TP + FP} \tag{3}$$

$$Recall = \frac{TP}{TP + FN} \tag{4}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{5}$$

where

- TP: A shapelet from the testing data that represents a drowsy state and is correctly predicted as drowsy by the classifier.

  - TN: A shapelet from the testing data that represents an awake state and is correctly predicted as awake by the classifier.
  - FP: A shapelet from the testing data that represents a drowsy state and is incorrectly predicted as awake by the classifier.
  - FN: A shapelet from the testing data that represents an awake state and is incorrectly predicted as drowsy by the classifier.

- Multi-Measure Drowsiness Detection: In this scenario, each classifier is applied to several measures at the same time. Algorithm 2 shows the process of multi-measure drowsiness detection, which takes as input the training shapelets of all the measures in $\mathcal{D}^{tr}$, except for EEG since it is not included in our experiments, along with the total number of shapelets (e.g., α) and the desired classifier. On the other hand, Algorithm 2 returns as output the driver's state, similarly to that of single-measure drowsiness detection. The algorithm process is divided into four steps: The first step, called multi-shapelet labeling (lines 1–10), monitors both driver and driving behaviors and concatenates the corresponding shapelets of all the measures into a single shapelet (line 3). Then, the concatenated shapelet is assigned a label of 1 if all the measures indicate a drowsy state and 0 otherwise, before being added to the labeled shapelet list (line 9). The second step, called data classification (lines 11–15), applies each classifier to the concatenated shapelets in the testing data (line 13) while adding the predicted state of each shapelet to the list of the predicted ones (line 14). The third and fourth steps, called classification performance (line 16) and driver-state determination (lines 17–23), compute the performance metrics of the classifier and return the driver's state, similar to the process described in Algorithm 1.

Algorithm 2 Multi-Measure Drowsiness Detection Algorithm

Require: Set of measures: $\mathcal{M} - \{EEG\}$; Training data: $\mathcal{D}^{tr}$; Testing data: $\mathcal{D}^{te}$; Number of total shapelets of each measure: α; A classifier: C.
Ensure: Driver state: awake or drowsy.

1: $\mathcal{L} \leftarrow \emptyset$
2: **for** $i = 1$ to α **do**
3: $\quad H_i^{1\to4} = H_i^1 \oplus H_i^2 \oplus H_i^3 \oplus H_i^4$
4: $\quad$ **if** $L_i^1 = 1$ and $L_i^2 = 1$ and $L_i^3 = 1$ and $L_i^4 = 1$ **then**
5: $\quad\quad L_i^{1\to4} = 1$
6: $\quad$ **else**
7: $\quad\quad L_i^{1\to4} = 0$
8: $\quad$ **end if**
9: $\quad \mathcal{L} \leftarrow \mathcal{L} \cup \{(H_i^{1\to4}, L_i^{1\to4})\}$
10: **end for**
11: $states_set \leftarrow \emptyset$
12: **for** each $(H_k^{1\to4}, L_k^{1\to4}) \in \mathcal{L}$ where $H_k^{1\to4} \in \mathcal{D}^{te}$ **do**
13: $\quad predicted_L_k^{1\to4} =$ apply_classifier $(C, H_k^{1\to4}, \mathcal{D}^{tr})$
14: $\quad states_set \leftarrow states_set \cup \{predicted_L_k^{1\to4}\}$
15: **end for**
16: [accuracy, precision, recall, F1] = confusion_matrix $(states_set, \mathcal{D}^{te})$
17: **for** each $predicted_L^{1\to4} \in states_set$ **do**
18: $\quad$ **if** $predicted_L^{1\to4} = 0$ **then**
19: $\quad\quad$ **return** awake
20: $\quad$ **else**
21: $\quad\quad$ **return** drowsy
22: $\quad$ **end if**
23: **end for**

3.3. Driver Notification and Database Updating

After determining the driver's state based on single- or multi-measures, a notification is sent to the driver only when a drowsiness state is detected. Subsequently, a notification may take several forms, including a signal to take a timely break, a coffee break, an alarm, the activation of music, etc. On the other hand, drivers differ visually and behaviorally while driving. For instance, a driver who makes several successive deviations may be experiencing inattention (e.g., a drowsy or distracted state), whereas the same behavior may indicate an alert driver intending to overtake other vehicles. Thus, the database stored in the vehicle may not fit the behavior of the driver who purchased it. To overcome this problem, we proposed dynamically updating the database according to the vehicle's owner. Hence, when a drowsiness state is detected, we proposed replacing the current shapelet with the most similar one, e.g., the one having the highest Jaccard similarity score, saved in the database. Consequently, after a certain driving period, the updated database will more accurately reflect the driver's individual behavior.

4. Performance Evaluation

In order to evaluate the effectiveness of RDDF, we used experiments with both locally executed tests and publicly available datasets as explained in Section 3.1.1. We used a laptop equipped with a 64-bit six-core Intel Core i7-4800MQ processor running at 2.7 GHz. In addition, the system was configured with 16 GB of RAM and a storage capacity of 512 GB. The machine ran on Windows 10, and the framework used in the implementation was Python 3.10. Furthermore, we fixed the percentage of training data β to 90%, and the remaining portion was used for testing data. Furthermore, to assess model generalization, a Leave-One-Subject-Out (LOSO) cross-validation scheme was additionally applied. Model performance was averaged across subjects and reported as mean ± standard deviation over five random data splits.

4.1. Rationale for T Selection

In our simulation, we selected window lengths $T = \{2, 3, 4\}$ seconds for the experiments. The selection was driven by practical considerations: windows shorter than 2 s reduce the number of samples per window and can miss gradual drowsiness patterns, while windows longer than 4 s increase the detection latency and decrease the responsiveness required for in-vehicle alerts. In addition, for the EEG data collected using consumer headsets with a high sampling rate, a 2–4 s window provides enough samples to compute stable summary statistics (mean and standard deviation) used in our shapelet-based features while being computationally feasible for on-board processing.

4.2. Performance Study of RDDF

In this section, we studied the effectiveness of RDDF in terms of three metrics: behavior type, classifier performance, and window length. Table 2 shows the performance metrics (e.g., accuracy, precision, recall, and F1) of each classifier when applying our technique to each behavioral measure individually or to all the measures simultaneously (e.g., column $\{SWA, PP, VS, TAV\}$). The obtained results are highly dependent on the window length (T), and several noteworthy observations can be made:

- In the best case, RDDF ensures up to 99.8% accuracy, 99% precision, 100% recall, and 99% F1-score. Otherwise, in the worst case, RDDF drops to 43% accuracy, 3% precision, <1% recall, and <1% F1-score. This decrease occurs because certain classifiers are not suitable for the given dataset characteristics. Across ten repeated data splits, the standard deviation was ±0.42%, confirming consistent model performance with low variance despite the limited sample size.
- Classifiers exhibit varying performance depending on each measure and the window length T. Indeed, we observed that there is no classifier that outperforms all the others across all the performance metrics. However, JKNN achieves superior classification results in most cases compared to the other classifiers. This is because the Jaccard similarity acts as a strict constraint, ensuring minimal classification error compared with the conditions used in other classifiers. Thus, it is unlikely that a shapelet labeled as drowsy would be similar to one labeled as awake according to the Jaccard similarity. Moreover, JKNN is generally followed by the RF classifier in performance metrics, mainly due to the large number of trees used in RF (e.g., 100). Subsequently, the classifiers are ordered according to the average accuracy as follows: JKNN (97.6%), random forest (91.1%), K-nearest neighbor (89.5%), decision tree (89.1%), logistic regression (88.5%), multi-layer perceptron (87.9%), and naïve Bayes (78.1%).
- The performance of all the classifiers is relatively stable when varying the window length. On the one hand, this indicates that our technique can provide fast and accurate drowsiness detection even with a short driver monitoring period (e.g., $T = 2$ s). On the other hand, this implies that the technique can reduce the frequency of driver alerts when using a longer monitoring period (e.g., $T = 4$ s).
- The performance of each classifier varies dynamically according to the monitored measure. First, the performance of a classifier changes depending on whether a single or multiple measure is monitored. For instance, the average accuracy of RF reaches 95.8% for each separate measure (SWA, PP, VS, and TAV), while it increases to 98% when all the measures are combined (i.e., SWA, PP, VS, and TAV). Second, the performance of the classifier also varies according to the type of measure, either driver or driving behaviors. For instance, the average accuracy of RF achieves 95.5% for driving-based measures (e.g., SWA, PP, and VS) and 76.6% for driver's-based measures (e.g., TAV and EEG). This is mainly because the shapelets representing driver's behavior vary significantly compared to those of driving behavior.
- The recall metric in JKNN is consistently equal to 1. This is because a shapelet indicating the awake state is highly unlikely to resemble that of the drowsy state according to the Jaccard similarity. Thus, FN is always equal to 0, and the recall metric correspondingly equals 1.
- The obtained results of precision, recall, and F1-score metrics of JKNN outperform those of conventional classifiers. This is due to the imbalanced data problem facing standard classifiers, which is effectively resolved in the new version of the classifier (e.g., JKNN); in fact, JKNN mostly focuses on the drowsy shapelets to classify the testing data independently of its size. Subsequently, we can notice that precision, recall, and F1 metrics mostly exceed 81% in JKNN while they mostly remain below 50% using standard classifiers.

Table 2. Performance evaluation of RDDF.

		SWA				PP				VS				TAV				{SWA, PP, VS, TAV}				EEG			
		Acc.	**Pre.**	**Rec.**	**F1**	**Acc.**	**Pre.**	**Rec.**	**F1**	**Acc.**	**Pre.**	**Rec.**	**F1**	**Acc.**	**Pre.**	**Rec.**	**F1**	**Acc.**	**Pre.**	**Rec.**	**F1**	**Acc.**	**Pre.**	**Rec.**	**F1**
$T = 2$ s	LR	0.94	0.88	0.08	0.14	0.91	0.5	0.007	0.01	0.96	1	0.01	0.03	0.98	0.63	0.21	0.31	0.94	0.76	0.17	0.28	0.57	0.6	0.6	0.6
	RF	0.99	0.99	0.98	0.98	0.91	0.37	0.04	0.07	0.96	0.33	0.01	0.03	0.98	0.66	0.31	0.42	0.99	0.94	0.97	0.96	0.66	0.7	0.65	0.67
	DT	0.99	0.96	0.98	0.97	0.84	0.17	0.17	0.17	0.92	0.08	0.09	0.09	0.98	0.95	0.43	0.57	0.99	0.93	0.98	0.95	0.63	0.65	0.62	0.63
	NB	0.94	0.54	1	0.7	0.69	0.06	0.17	0.09	0.72	0.06	0.4	0.1	0.98	0.55	0.23	0.32	0.94	0.55	1	0.71	0.53	0.53	0.89	0.67
	KNN	0.98	0.93	0.92	0.92	0.89	0.44	0.36	0.39	0.96	0.51	0.25	0.33	0.98	0.66	0.3	0.42	0.96	0.81	0.65	0.72	0.59	0.62	0.6	0.61
	MLP	0.99	0.97	0.89	0.93	0.9	0.33	0.02	0.03	0.96	-	0	-	0.98	0.36	0.24	0.35	0.93	0.5	0	0.01	0.53	0.6	0.4	0.48
	JKNN	0.99	0.97	1	0.98	0.99	0.98	1	0.99	0.99	0.81	1	0.9	0.98	0.95	1	0.97	0.98	0.81	1	0.89	0.99	0.94	1	0.97
$T = 3$ s	LR	0.95	0.84	0.18	0.3	0.93	-	0	-	0.96	0.25	0.02	0.05	0.99	0.6	0.48	0.54	0.95	0.76	0.16	0.27	0.52	0.53	0.6	0.56
	RF	0.98	0.91	0.88	0.89	0.92	0.33	0.01	0.02	0.95	0.5	0.02	0.04	0.97	0.61	0.27	0.38	0.98	0.9	0.87	0.88	0.65	0.69	0.62	0.65
	DT	0.98	0.92	0.9	0.91	0.87	0.15	0.14	0.14	0.94	0.05	0.06	0.06	0.98	0.91	0.39	0.55	0.99	0.86	0.96	0.9	0.59	0.62	0.58	0.6
	NB	0.93	0.52	1	0.68	0.43	0.05	0.5	0.09	0.74	0.05	0.44	0.09	0.98	0.53	0.23	0.32	0.94	0.56	1	0.72	0.53	0.53	0.88	0.66
	KNN	0.98	0.89	0.9	0.89	0.9	0.36	0.25	0.29	0.94	0.43	0.22	0.29	0.98	0.63	0.3	0.41	0.96	0.72	0.58	0.64	0.61	0.61	0.66	0.64
	MLP	0.99	0.92	0.91	0.92	0.93	0.2	0.01	0.02	0.96	-	0	-	0.98	0.6	0.24	0.35	0.93	-	0	-	0.46	0.5	0.13	0.21
	JKNN	0.99	0.91	1	0.95	0.99	0.9	1	0.95	0.99	0.8	1	0.88	0.99	0.89	1	0.94	0.99	0.8	1	0.88	0.98	0.92	1	0.96
$T = 4$ s	LR	0.95	0.75	0.19	0.3	0.91	-	0	-	0.96	0.33	0.03	0.06	0.98	0.53	0.5	0.51	0.94	0.83	0.24	0.38	0.6	0.62	0.6	0.61
	RF	0.98	0.91	0.8	0.85	0.93	0.5	0.03	0.06	0.97	0.5	0.09	0.15	0.96	0.61	0.25	0.36	0.98	0.92	0.81	0.86	0.65	0.7	0.63	0.66
	DT	0.99	0.95	0.87	0.91	0.86	0.12	0.12	0.12	0.94	0.06	0.1	0.07	0.98	0.94	0.38	0.54	0.99	0.9	0.95	0.93	0.59	0.62	0.55	0.58
	NB	0.92	0.49	1	0.66	0.71	0.06	0.25	0.1	0.7	0.03	0.23	0.05	0.96	0.5	0.23	0.32	0.93	0.46	1	0.63	0.5	0.51	0.88	0.65
	KNN	0.98	0.84	0.77	0.81	0.91	0.46	0.27	0.34	0.95	0.28	0.18	0.22	0.98	0.6	0.25	0.29	0.96	0.79	0.41	0.54	0.61	0.64	0.69	0.66
	MLP	0.98	0.83	0.85	0.84	0.91	0.25	0.04	0.08	0.96	-	0	-	0.97	0.54	0.23	0.32	0.95	0.63	0.63	0.45	0.52	0.54	0.68	0.6
	JKNN	0.98	0.86	1	0.92	0.99	0.85	1	0.91	0.98	0.73	1	0.84	0.99	0.55	1	0.71	0.98	0.55	1	0.71	0.98	0.9	1	0.95

4.3. Training Time Analysis

Table 3 shows the training time of each classifier used in the RDDF framework for the same hardware configuration. The results demonstrate that the training time increases consistently with the window length T, since larger windows produce higher-dimensional feature vectors and more training samples. Among conventional classifiers, naïve Bayes and decision tree require the least time to train (below two seconds for all T values), reflecting their simple parameter estimation and tree construction procedures. logistic regression and random forest exhibit moderate training costs, scaling nearly linearly with T. KNN shows negligible training time because the method performs no model fitting; only an optional distance-indexing step is included in the timing. In contrast, the proposed JKNN demands noticeably higher training effort due to the pairwise Jaccard similarity computation between all the training samples. Even so, its training time remains under one minute for the largest window ($T = 4$ s), indicating its practical feasibility for onboard or near-real-time applications. The multi-layer perceptron (MLP) requires the longest training time because of its iterative gradient-based optimization; however, the observed increase with T is still roughly proportional to data size, confirming its stable computational scalability. Overall, the measured results demonstrate that while the JKNN method incurs a higher computational cost than the classical distance-based KNN, it provides a reasonable trade-off between accuracy and efficiency, especially for medium window durations ($T = 2$–3 s), where both runtime and recognition performance are balanced.

Table 3. Average training time (seconds) for each classifier and window length.

Classifier	T= 2 s	T = 3 s	T = 4 s
Logistic Regression (LR)	2.9	4.2	6.0
Decision Tree (DT)	0.8	1.1	1.6
Random Forest (RF)	8.7	12.6	17.8
Naïve Bayes (Gaussian NB)	0.6	0.9	1.3
K-Nearest Neighbors (KNN, K = 5)	0.2	0.3	0.5
Multi-Layer Perceptron (MLP, 500/250/40 neurons)	68.3	95.4	132.1
Jaccard-KNN (JKNN, K = 5)	27.0	38.2	52.4

4.4. RDDF vs. State-of-the-Art: An Analytical Comparison

Table 4 compares the performance of RDDF to several state-of-the-art techniques. The table shows the best accuracy results obtained with each technique while summarizing the publication year, employed models, monitored behavior type, monitored measures, and corresponding accuracy. The following observations can be made based on the table:

- Most of the proposed techniques are based on machine- and deep-learning methods.
- The literature shows significant progress in addressing the drowsiness detection problem, with reported accuracies mostly exceeding 90%, except for [24].
- Researchers focus equally on driving and driver behaviors for detecting drowsy drivers, and both driving and physiological measures have been extensively investigated in the literature.
- RDDF is well-positioned in terms of accuracy and shows strong performance compared to other existing techniques. This is mainly because our technique treats each record within a shapelet as an individual feature when classifying the driver's state. Thus, it outperforms most existing approaches, except for one technique (e.g., [18]) that achieves slightly higher accuracy than RDDF.

Table 4. Analytical comparison between RDDF and state-of-the-art techniques.

Reference	Year	Technique	Monitored Behavior	Measures	Accuracy
Wang et al. [15]	2017	SVM	Driving	VS	95%
Dehzangi et al. [16]	2018	DT + Ensemble	Driving	Acceleration, Braking, SWM	99.10%
Arefnezhad et al. [17]	2019	Fuzzy inference + SVM	Driving	SWM	98.12%
Ma et al. [20]	2019	PCANet + SVM + KNN	Driver	EEG	95%
Luo et al. [21]	2019	ASF + Entropy	Driver	EEG	95.37%
Budak et al. [22]	2019	LSTM	Driver	EEG	94.31%
De Naurois et al. [23]	2019	CNN	Driving and Driver	Heart rate, head movement, EM, VLD, VS, SWM	95%
Shahverdy et al. [18]	2020	CNN	Driving	Gravity, VS, throttle, acceleration	99.99%
Jiao et al. [19]	2020	WT + LSTM	Driver	EEG, EOG	98%
Zhang et al. [24]	2020	Logit function	Driving and Driver	VLD, EM	62.84%
This work	2025	RDDF	Driving and Driver	SWA, PP, VS, TAV, EEG	99.8%

4.5. RDDF vs. State-of-the-Art: An Empirical Comparison

To ensure a fair and reproducible comparison, we conducted ten independent runs of each model under identical experimental conditions. The same dataset, preprocessing pipeline, and train–test partitioning used for evaluating RDDF were also applied to the baseline techniques of [18,23]. All the hyperparameters were kept consistent with the authors' original configurations whenever available, and random seeds were varied across runs to minimize bias. The mean values of accuracy, precision, recall, and F1-score over these ten runs are reported in Table 5. This approach eliminates the influence of dataset or environmental differences, allowing the comparison to focus solely on the intrinsic modeling capability of each technique.

As summarized in Table 5, the proposed RDDF framework consistently outperforms both benchmark techniques across all the evaluation metrics. RDDF achieved an average accuracy of 95.8%, surpassing [23] (92.0%) and [18] (90.5%) by margins of approximately 3–5%. Similar improvements are observed in precision, recall, and F1-score, confirming that RDDF provides a more balanced and reliable detection of drowsiness events. These gains can be attributed to the hybrid integration of recurrent dynamic features and discriminative shapelets, which enable the model to capture both temporal evolution and statistical variability. It is also worth noting that the differences between our reproduced results and those originally reported by the respective authors are relatively small, indicating that the re-runs on our simulated dataset yield consistent trends and further confirm the robustness of the comparative analysis.

Table 5. Empirical comparison between RDDF and state-of-the-art techniques.

Reference	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
De Naurois et al. [23]	92.0 ± 0.9	91.5 ± 0.6	90.8 ± 0.7	91.1 ± 0.6
Shahverdy et al. [18]	90.5 ± 1.1	90.0 ± 1.06	89.7 ± 0.97	89.9 ± 0.99
RDDF	95.8 ± 0.42	95.4 ± 0.57	95.1 ± 0.61	95.2 ± 0.58

4.6. RDDF Limitations

Although the proposed framework demonstrates significant efficiency in accurately detecting driver drowsiness rapidly, certain limitations remain that prevent a full validation of the RDDF's potential in real-world scenarios. In particular, we tested $T = 2$, 3, and 4 s to balance detection responsiveness and temporal feature capture. We acknowledged that broader sweeps of T and sliding/overlapping window strategies could reveal additional trade-offs. We, therefore, listed a targeted set of experiments for future work: (i) evaluate T across 1–10 s in 1-s steps for all metrics and measures; (ii) compare non-overlapping vs. sliding windows (50% overlap); and (iii) test adaptive windowing methods and online change-point detectors for lower latency. These directions are expected to yield further improvements and will be pursued in follow-up work.

Second, the JKNN algorithm, while achieving high accuracy, exhibits increased computational complexity when scaling to large datasets or multi-modal signals. As a future direction, we aim to expand the dataset to include multi-channel EEG and complementary behavioral data (e.g., facial or steering signals), develop hybrid fusion approaches to handle multi-modal information, and implement algorithmic optimizations or parallelization strategies to enhance the real-time efficiency of JKNN. Addressing these aspects will further improve the robustness, scalability, and practical applicability of RDDF.

5. Conclusions and Future Work

In this study, a drowsiness detection framework was developed by integrating sensing technologies with supervised learning algorithms. The proposed JKNN-based framework

was preliminarily validated using real sensor data, showing a high level of accuracy in detecting driver drowsiness. Through extensive experiments, an accuracy of 99.0% was achieved on the collected dataset, and 96.8% on a public dataset, confirming the effectiveness and adaptability of the proposed approach. Comparative analyses with conventional classifiers demonstrated that the JKNN model achieved superior detection performance while maintaining computational efficiency. The impact of varying window lengths and training parameters was also examined, and the results indicated a consistent trend in both accuracy and training time. The findings suggest that driver drowsiness can be effectively detected using a combination of physiological and behavioral sensing data processed by lightweight supervised models. Although the obtained results indicate the feasibility of the framework, further validation using larger and multi-channel datasets is required to confirm robustness and improve generalizability. In future work, we plan to explore real-time deployment and adaptive calibration under dynamic driving conditions to enhance system reliability and user safety.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The author declares no conflicts of interest.

References

1. El Zein, H.; Harb, H.; Delmotte, F.; Zahwe, O.; Haddad, S. VAS-3D: A Visual-Based Alerting System for Detecting Drowsy Drivers in Intelligent Transportation Systems. *World Electr. Veh. J.* **2024**, *15*, 540. [CrossRef]
2. Essahraui, S.; Lamaakal, I.; El Hamly, I.; Maleh, Y.; Ouahbi, I.; El Makkaoui, K.; Filali Bouami, M.; Pławiak, P.; Alfarraj, O.; Abd El-Latif, A.A. Real-time driver drowsiness detection using facial analysis and machine learning techniques. *Sensors* **2025**, *25*, 812. [CrossRef] [PubMed]
3. Ahmad Jan, M.; Adil, M.; Brik, B.; Harous, S.; Abbas, S. Making Sense of Big Data in Intelligent Transportation Systems: Current Trends, Challenges and Future Directions. *ACM Comput. Surv.* **2025**, *57*, 197. [CrossRef]
4. Ganguly, B.; Dey, D.; Munshi, S. An Attention Deep Learning Framework-Based Drowsiness Detection Model for Intelligent Transportation System. *IEEE Trans. Intell. Transp. Syst.* **2025**, *26*, 4517–4527. [CrossRef]
5. Jaulkar, S.; Parihar, A. Different types of injury associated with road traffic accidents. *Multidiscip. Rev.* **2025**, *8*, e2025111. [CrossRef]
6. Soares, S.; Monteiro, T.; Lobo, A.; Couto, A.; Cunha, L.; Ferreira, S. Analyzing driver drowsiness: From causes to effects. *Sustainability* **2020**, *12*, 1971. [CrossRef]
7. Cao, S.; Feng, P.; Kang, W.; Chen, Z.; Wang, B. Optimized driver fatigue detection method using multimodal neural networks. *Sci. Rep.* **2025**, *15*, 12240. [CrossRef]
8. Khanna, M.; Nossam, S.C.; Pulastya, G.; Bandi, H.R.; Anirudh, R. Real-time Driver Drowsiness Detection for safer driving using MESA: A multi-agent based modeling framework. *AIP Conf. Proc.* **2025**, *3237*, 020002.
9. Venkatesan, T.; Naveena, S.; Aadhish, M.; Nandha, S. A Smart IoT-Based Framework for Adaptive Drowsiness Detection and Alert Mechanisms for Preventing Fatigue-Induced Accidents. In Proceedings of the 2025 3rd International Conference on Artificial Intelligence and Machine Learning Applications Theme: Healthcare and Internet of Things (AIMLA), Tiruchengode, India, 29–30 April 2025; IEEE: Piscataway, NJ, USA, 2025; pp. 1–4.
10. Yang, E.; Yi, O. Enhancing road safety: Deep learning-based intelligent driver drowsiness detection for advanced driver-assistance systems. *Electronics* **2024**, *13*, 708. [CrossRef]
11. Jarndal, A.; Tawfik, H.; Siam, A.I.; Alsyouf, I.; Cheaitou, A. A real-time vision transformers-based system for enhanced driver drowsiness detection and vehicle safety. *IEEE Access* **2024**, *13*, 1790–1803. [CrossRef]
12. Doudou, M.; Bouabdallah, A.; Berge-Cherfaoui, V. Driver drowsiness measurement technologies: Current research, market solutions, and challenges. *Int. J. Intell. Transp. Syst. Res.* **2020**, *18*, 297–319. [CrossRef]

13. El-Nabi, S.A.; El-Shafai, W.; El-Rabaie, E.S.M.; Ramadan, K.F.; Abd El-Samie, F.E.; Mohsen, S. Machine learning and deep learning techniques for driver fatigue and drowsiness detection: A review. *Multimed. Tools Appl.* **2024**, *83*, 9441–9477. [CrossRef]
14. Lacherre, J.; Castillo-Sequera, J.L.; Mauricio, D. Factors, prediction, and explainability of vehicle accident risk due to driving behavior through machine learning: A systematic literature review, 2013–2023. *Computation* **2024**, *12*, 131. [CrossRef]
15. Wang, D.; Pei, X.; Li, L.; Yao, D. Risky driver recognition based on vehicle speed time series. *IEEE Trans. Hum.-Mach. Syst.* **2017**, *48*, 63–71. [CrossRef]
16. Dehzangi, O.; Masilamani, S. Unobtrusive driver drowsiness prediction using driving behavior from vehicular sensors. In Proceedings of the 2018 24th International Conference on Pattern Recognition (ICPR), Beijing, China, 20–24 August 2018; pp. 3598–3603.
17. Arefnezhad, S.; Samiee, S.; Eichberger, A.; Nahvi, A. Driver drowsiness detection based on steering wheel data applying adaptive neuro-fuzzy feature selection. *Sensors* **2019**, *19*, 943. [CrossRef]
18. Shahverdy, M.; Fathy, M.; Berangi, R.; Sabokrou, M. Driver behavior detection and classification using deep convolutional neural networks. *Expert Syst. Appl.* **2020**, *149*, 113240. [CrossRef]
19. Jiao, Y.; Deng, Y.; Luo, Y.; Lu, B.L. Driver sleepiness detection from EEG and EOG signals using GAN and LSTM networks. *Neurocomputing* **2020**, *408*, 100–111. [CrossRef]
20. Ma, Y.; Chen, B.; Li, R.; Wang, C.; Wang, J.; She, Q.; Luo, Z.; Zhang, Y. Driving fatigue detection from EEG using a modified PCANet method. *Comput. Intell. Neurosci.* **2019**, *2019*, 4721863. [CrossRef]
21. Luo, H.; Qiu, T.; Liu, C.; Huang, P. Research on fatigue driving detection using forehead EEG based on adaptive multi-scale entropy. *Biomed. Signal Process. Control* **2019**, *51*, 50–58. [CrossRef]
22. Budak, U.; Bajaj, V.; Akbulut, Y.; Atila, O.; Sengur, A. An effective hybrid model for EEG-based drowsiness detection. *IEEE Sens. J.* **2019**, *19*, 7624–7631. [CrossRef]
23. de Naurois, C.J.; Bourdin, C.; Stratulat, A.; Diaz, E.; Vercher, J.L. Detection and prediction of driver drowsiness using artificial neural network models. *Accid. Anal. Prev.* **2019**, *126*, 95–104. [CrossRef]
24. Zhang, X.; Wang, X.; Yang, X.; Xu, C.; Zhu, X.; Wei, J. Driver drowsiness detection using mixed-effect ordered logit model considering time cumulative effect. *Anal. Methods Accid. Res.* **2020**, *26*, 100114. [CrossRef]
25. Wang, J.; Cai, M.; Zhu, Z.; Ding, H.; Yi, J.; Du, A. VTD: Visual and Tactile Database for Driver State and Behavior Perception. *arXiv* **2024**, arXiv:2412.04888. [CrossRef]
26. Bouaziz, B. Drowsiness Detection Dataset. 2020. Available online: https://github.com/bassem-bouaziz/Drowsiness_Detection (accessed on 7 January 2023).
27. Cramer, J.S. *The Origins of Logistic Regression*; Tinbergen Institute: Amsterdam/Rotterdam, The Netherlands , 2002; pp. 167–178.
28. Yan, L.; Huang, Z.; Zhang, Y.; Zhang, L.; Zhu, D.; Ran, B. Driving risk status prediction using Bayesian networks and logistic regression. *IET Intell. Transp. Syst.* **2017**, *11*, 431–439. [CrossRef]
29. Murata, A. Proposal of a method to predict subjective rating on drowsiness using physiological and behavioral measures. *IIE Trans. Occup. Ergon. Hum. Factors* **2016**, *4*, 128–140. [CrossRef]
30. Parmar, A.; Katariya, R.; Patel, V. A review on random forest: An ensemble classifier. In *Proceedings of the International Conference on Intelligent Data Communication Technologies and Internet of Things*; Springer: Cham, Switzerland, 2018; pp. 758–763.
31. Gwak, J.; Hirao, A.; Shino, M. An investigation of early detection of driver drowsiness using ensemble machine learning based on hybrid sensing. *Appl. Sci.* **2020**, *10*, 2890. [CrossRef]
32. Tao, Q.; Zhang, R.; Yan, Y.; Zhang, C.; Li, Z. Improvement of random forest cascade regression algorithm and its application in fatigue detection. In Proceedings of the 2019 IEEE 2nd International Conference on Electronics Technology (ICET), Chengdu, China, 10–13 May 2019; pp. 499–503.
33. Charbuty, B.; Abdulazeez, A. Classification based on decision tree algorithm for machine learning. *J. Appl. Sci. Technol. Trends* **2021**, *2*, 20–28. [CrossRef]
34. Hu, J.; Min, J. Automated detection of driver fatigue based on EEG signals using gradient boosting decision tree model. *Cogn. Neurodynamics* **2018**, *12*, 431–440. [CrossRef]
35. Yimyam, W.; Ketcham, M. The System for Driver Fatigue Monitoring Using Decision Tree via Wireless Sensor Network for Intelligent Transport System. *Int. J. Online Eng.* **2018**, *14*, 21–39. [CrossRef]
36. Yao, Y.; Zhao, X.; Du, H.; Zhang, Y.; Zhang, G.; Rong, J. Classification of fatigued and drunk driving based on decision tree methods: A simulator study. *Int. J. Environ. Res. Public Health* **2019**, *16*, 1935. [CrossRef]
37. Hand, D.J.; Yu, K. Idiot's Bayes—Not so stupid after all? *Int. Stat. Rev.* **2001**, *69*, 385–398.
38. Bakheet, S.; Al-Hamadi, A. A Framework for Instantaneous Driver Drowsiness Detection Based on Improved HOG Features and Naïve Bayesian Classification. *Brain Sci.* **2021**, *11*, 240. [CrossRef]
39. Nsaif, A.K.; Ali, S.H.M.; Jassim, K.N.; Nseaf, A.K.; Sulaiman, R.; Al-Qaraghuli, A.; Wahdan, O.; Nayan, N.A. FRCNN-GNB: Cascade Faster R-CNN With Gabor Filters and Naïve Bayes for Enhanced Eye Detection. *IEEE Access* **2021**, *9*, 15708–15719. [CrossRef]

40. Abu Alfeilat, H.A.; Hassanat, A.B.; Lasassmeh, O.; Tarawneh, A.S.; Alhasanat, M.B.; Eyal Salman, H.S.; Prasath, V.S. Effects of distance measure choice on k-nearest neighbor classifier performance: A review. *Big Data* **2019**, *7*, 221–248. [CrossRef] [PubMed]
41. Li, Z.; Zhang, Q.; Zhao, X. Performance analysis of K-nearest neighbor, support vector machine, and artificial neural network classifiers for driver drowsiness detection with different road geometries. *Int. J. Distrib. Sens. Netw.* **2017**, *13*, 1550147717733391. [CrossRef]
42. Ekaputri, C.; Fu'adah, Y.N.; Pratiwi, N.K.C.; Rizal, A.; Sularso, A.N. Drowsiness Detection Based on EEG Signal Using Discrete Wavelet Transform (DWT) and K-Nearest Neighbors (K-NN) Methods. In *Proceedings of the 1st International Conference on Electronics, Biomedical Engineering, and Health Informatics*; Springer: Singapore, 2021; pp. 487–498.
43. Hemalatha, K.; Rani, K.U. Advancements in multi-layer perceptron training to improve classification accuracy. *Int. J. Recent Innov. Trends Comput. Commun.* **2017**, *5*, 353–357.
44. Chou, C.L.; Huang, Y.H.; Ho, S.C. Blink Detection Using Facial Landmark Blink Detector and Multi-Layer Perceptron. In Proceedings of the NCS 2019, Huntsville, AL, USA, 4–6 June 2019; pp. 542–545.
45. Zou, B.; Shen, M.; Li, X.; Zheng, Y.; Zhang, L. EEG-based driving fatigue detection during operating the steering wheel data section. In Proceedings of the 2020 42nd Annual International Conference of the IEEE Engineering in Medicine & Biology Society (EMBC), Montreal, QC, Canada, 20–24 July 2020; pp. 248–251.
46. Guo, K.; He, L.; Chen, Y.; Guo, W.; Zheng, J. A local community detection algorithm based on internal force between nodes. *Appl. Intell.* **2020**, *50*, 328–340. [CrossRef]
47. Diana, N.E.; Ulfa, I.H. Measuring Performance of N-Gram and Jaccard-Similarity Metrics in Document Plagiarism Application. *J. Physics Conf. Ser.* **2019**, *1196*, 012069. [CrossRef]
48. Li, B.; Gao, M.; Ma, L.; Liang, Y.; Chen, G. Web application-layer DDoS attack detection based on generalized Jaccard similarity and information entropy. In *Proceedings of the International Conference on Artificial Intelligence and Security*; Springer: Cham, Switzerland, 2019; pp. 576–585.
49. Moodley, R.; Chiclana, F.; Caraffini, F.; Carter, J. Application of uninorms to market basket analysis. *Int. J. Intell. Syst.* **2019**, *34*, 39–49. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.